Методическое пособие:
Создание Flutter-приложения с авторизацией пользователей

Дисциплина: 3306.ФЛТ.26В Flutter
Контрольная точка: КТ № 5 — Создание моделей пользователя
Автор: Максаков Михаил, преподаватель колледжа ITHUB

Цель методички

Данное пособие поможет студентам шаг за шагом создать Flutter-приложение с авторизацией пользователей и локальным сохранением данных. После изучения материала студенты смогут:

- Создавать модели данных в Dart
- Реализовывать навигацию между экранами
- Управлять состоянием приложения
- Сохранять и загружать данные пользователя

Что мы создаем

Мы создадим приложение, которое:

1. При первом запуске показывает экран авторизации
2. После успешного входа сохраняет данные локально
3. При повторном запуске сразу открывает профиль пользователя
4. Позволяет выйти из аккаунта с очисткой данных

Этапы разработки

Этап 1: Создание проекта и установка зависимостей
- 1.1 Создаем новый Flutter проект
- 1.2 Переходим в папку проекта
- 1.3 Открываем проект в VS Code
- 1.4 Добавляем зависимость shared_preferences
- 1.5 Устанавливаем зависимости

Этап 2: Создание структуры папок
Этап 3: Создание модели пользователя (UserModel)
Этап 4: Создание сервиса для работы с хранилищем
Этап 5: Создание экрана авторизации
Этап 6: Создание экрана профиля
Этап 7: Главный файл приложения (main.dart)
Этап 8: Запуск и тестирование
- 8.1 Запуск приложения
- 8.2 Тест-кейсы для проверки

Этап 1: Создание проекта и установка зависимостей

1.1 Создаем новый Flutter проект

Откройте терминал и выполните команду:

```bash
flutter create user_app
```

Эта команда создаст папку `user_app` со всеми необходимыми файлами.

1.2 Переходим в папку проекта

```bash
cd user_app
```

1.3 Открываем проект в VS Code

```bash
code .
```

1.4 Добавляем зависимость `shared_preferences`

Откройте файл **`pubspec.yaml`** и в раздел `dependencies` добавьте строку:

```yaml
dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.2.2   # ДОБАВИТЬ ЭТУ СТРОКУ
```

1.5 Устанавливаем зависимости

```bash
flutter pub get
```

Мы добавили библиотеку `shared_preferences`, которая позволяет сохранять данные на устройстве пользователя.

Этап 2: Создание структуры папок

Организуем код, чтобы он был чистым и понятным. В папке `lib` создаем следующие подпапки:

```
lib/
├── models/      # Здесь будут модели данных
├── screens/     # Здесь будут экраны приложения
└── services/    # Здесь будут сервисы (работа с данными)
```

Как создать в VS Code:**
- Клик правой кнопкой на папке `lib`
- Выбрать `New Folder`
- Создать: `models`, `screens`, `services`

Этап 3: Создание модели пользователя (UserModel)

Модель — это класс, который описывает структуру данных. В нашем случае пользователь имеет имя и email.

Создайте файл: `lib/models/user_model.dart`

```dart
// lib/models/user_model.dart
class UserModel {
  // Свойства класса (поля)
  final String name;
  final String email;

  // Конструктор (обязательные параметры)
  UserModel({
    required this.name,
    required this.email,
  });

  // Преобразование объекта в Map (для сохранения)
  Map<String, String> toJson() => {
      'name': name,
      'email': email,
    };

  // Создание объекта из Map (для загрузки)
  factory UserModel.fromJson(Map<String, String> json) {
    return UserModel(
      name: json['name'] ?? '',
      email: json['email'] ?? '',
    );
  }
}
```

Объяснение кода:
- `required` — означает, что параметр обязателен при создании объекта
- `toJson()` — превращает объект в Map, чтобы можно было сохранить
- `fromJson()` — создает объект из Map, когда загружаем сохраненные данные

Этап 4: Создание сервиса для работы с хранилищем

Сервис будет отвечать за сохранение, загрузку и удаление данных пользователя.

Создайте файл: `lib/services/storage_service.dart`

```dart
// lib/services/storage_service.dart
import 'package:shared_preferences/shared_preferences.dart';
import '../models/user_model.dart';

class StorageService {
  // Ключи для хранения (как имена переменных в хранилище)
  static const String _nameKey = 'user_name';
  static const String _emailKey = 'user_email';
  static const String _isLoggedInKey = 'is_logged_in';

  // Метод 1: Сохранение пользователя
  Future<void> saveUser(UserModel user) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_nameKey, user.name);
    await prefs.setString(_emailKey, user.email);
    await prefs.setBool(_isLoggedInKey, true);
  }

  // Метод 2: Загрузка пользователя
  Future<UserModel?> getUser() async {
    final prefs = await SharedPreferences.getInstance();
    final name = prefs.getString(_nameKey);
    final email = prefs.getString(_emailKey);

    if (name != null && email != null) {
      return UserModel(name: name, email: email);
    }
    return null; // Если данных нет
  }

  // Метод 3: Проверка авторизации
  Future<bool> isLoggedIn() async {
    final prefs = await SharedPreferences.getInstance();
```

```
    return prefs.getBool(_isLoggedInKey) ?? false;
  }

  // Метод 4: Выход из аккаунта (очистка)
  Future<void> logout() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.clear();
  }
}
```

Объяснение методов:
- `saveUser()` — сохраняет имя, email и ставит флаг "вошел"
- `getUser()` — загружает данные, если они есть
- `isLoggedIn()` — проверяет, авторизован ли пользователь
- `logout()` — полностью очищает все сохраненные данные

Этап 5: Создание экрана авторизации

Это первый экран, который видит пользователь при запуске.

Создайте файл: `lib/screens/auth_screen.dart`

```dart
// lib/screens/auth_screen.dart
import 'package:flutter/material.dart';
import '../models/user_model.dart';
import '../services/storage_service.dart';
import 'profile_screen.dart';

class AuthScreen extends StatefulWidget {
  const AuthScreen({super.key});

  @override
  State<AuthScreen> createState() => _AuthScreenState();
}

class _AuthScreenState extends State<AuthScreen> {
  // Ключ для формы (нужен для валидации)
  final _formKey = GlobalKey<FormState>();

  // Контроллеры для полей ввода
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();

  // Сервис для сохранения
  final _storage = StorageService();

  // Метод сохранения пользователя
  void _saveUser() async {
    if (_formKey.currentState!.validate()) {
      final user = UserModel(
        name: _nameController.text,
        email: _emailController.text,
      );
      await _storage.saveUser(user);

      if (mounted) {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => const ProfileScreen()),
```

```
      );
    }
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Авторизация'),
      centerTitle: true,
    ),
    body: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Form(
        key: _formKey,
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.person, size: 80, color: Colors.blue),
            const SizedBox(height: 30),

            // Поле для ввода имени
            TextFormField(
              controller: _nameController,
              decoration: const InputDecoration(
                labelText: 'Имя',
                border: OutlineInputBorder(),
                prefixIcon: Icon(Icons.person_outline),
              ),
              validator: (value) {
                if (value == null || value.isEmpty) {
                  return 'Введите имя';
                }
                return null;
              },
            ),

            const SizedBox(height: 15),

            // Поле для ввода email
            TextFormField(
              controller: _emailController,
              decoration: const InputDecoration(
```

```
            labelText: 'Email',
            border: OutlineInputBorder(),
            prefixIcon: Icon(Icons.email_outlined),
          ),
          validator: (value) {
            if (value == null || value.isEmpty) {
              return 'Введите email';
            }
            if (!value.contains('@')) {
              return 'Введите корректный email';
            }
            return null;
          },
        ),

        const SizedBox(height: 30),

        // Кнопка входа
        ElevatedButton(
          onPressed: _saveUser,
          style: ElevatedButton.styleFrom(
            minimumSize: const Size(double.infinity, 50),
          ),
          child: const Text('Войти'),
        ),
      ],
    ),
  ),
),
);
}

// Очистка контроллеров при закрытии экрана
@override
void dispose() {
  _nameController.dispose();
  _emailController.dispose();
  super.dispose();
}
}
```

Важные моменты:
- `Form` и `GlobalKey` — для валидации полей

- `TextFormField` с `validator` — проверяет, что поле не пустое и email содержит @
- `Navigator.pushReplacement` — заменяет текущий экран на новый (нельзя вернуться назад)

Этап 6: Создание экрана профиля

Экран, который показывает данные пользователя.

Создайте файл: `lib/screens/profile_screen.dart`

```dart
// lib/screens/profile_screen.dart
import 'package:flutter/material.dart';
import '../services/storage_service.dart';
import 'auth_screen.dart';

class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _storage = StorageService();
  String _name = '';
  String _email = '';

  // Загрузка данных при открытии экрана
  @override
  void initState() {
    super.initState();
    _loadUserData();
  }

  // Загрузка пользователя из хранилища
  Future<void> _loadUserData() async {
    final user = await _storage.getUser();
    if (user != null) {
      setState(() {
        _name = user.name;
        _email = user.email;
      });
    }
  }

  // Выход из аккаунта
  Future<void> _logout() async {
```

```
  await _storage.logout();
  if (mounted) {
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => const AuthScreen()),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Мой профиль'),
      centerTitle: true,
      actions: [
        IconButton(
          onPressed: _logout,
          icon: const Icon(Icons.logout),
          tooltip: 'Выйти',
        ),
      ],
    ),
    body: Center(
      child: Card(
        margin: const EdgeInsets.all(20),
        elevation: 4,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(15),
        ),
        child: Padding(
          padding: const EdgeInsets.all(25.0),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const CircleAvatar(
                radius: 50,
                backgroundColor: Colors.blue,
                child: Icon(Icons.person, size: 60, color: Colors.white),
              ),
              const SizedBox(height: 20),
              const Divider(),
              const SizedBox(height: 10),
              ListTile(
```

```
                leading: const Icon(Icons.person_outline),
                title: const Text('Имя'),
                subtitle: Text(
                  _name.isEmpty ? 'Не указано' : _name,
                  style: const TextStyle(
                    fontSize: 18,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
              ListTile(
                leading: const Icon(Icons.email_outlined),
                title: const Text('Email'),
                subtitle: Text(
                  _email.isEmpty ? 'Не указан' : _email,
                  style: const TextStyle(fontSize: 16),
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}
}
```

Ключевые моменты:

- `initState()` — вызывается при создании виджета, загружаем данные
- `setState()` — обновляет интерфейс после загрузки данных
- `Card` и `ListTile` — элементы Material Design для красивого отображения

Этап 7: Главный файл приложения (main.dart)

Объединяем все экраны и добавляем логику автопроверки.

Обновите файл: `lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'screens/auth_screen.dart';
import 'screens/profile_screen.dart';
import 'services/storage_service.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter User App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: true,
      ),
      debugShowCheckedModeBanner: false,
      home: FutureBuilder<bool>(
        future: StorageService().isLoggedIn(),
        builder: (context, snapshot) {
          // Пока проверяем статус - показываем загрузку
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Scaffold(
              body: Center(child: CircularProgressIndicator()),
            );
          }

          // Если пользователь авторизован - показываем профиль
          if (snapshot.data == true) {
            return const ProfileScreen();
          }
```

```
        // Иначе - экран авторизации
        return const AuthScreen();
      },
    ),
  );
 }
}
```

Объяснение `FutureBuilder`:
- Асинхронно проверяет, авторизован ли пользователь
- Пока проверка идет — показывает крутящийся индикатор
- После проверки открывает нужный экран

Этап 8: Запуск и тестирование

8.1 Запуск приложения

```bash
flutter run -d chrome
```

8.2 Тест-кейсы для проверки

Действие | Ожидаемый результат |
Первый запуск | Открывается экран авторизации |
Ввод имени и email, нажатие "Войти" | Открывается профиль с введенными данными |
Закрыть и снова открыть приложение | Сразу открывается профиль (данные сохранились) |
Нажать иконку выхода | Открывается экран авторизации |
Ввести пустое имя | Появляется ошибка "Введите имя" |
Ввести email без @ | Появляется ошибка "Введите корректный email" |